

Heads at a distance

The dyadic machine of Expo 1 prices space by grade and movement by the tree metric, but it treats all heads alike: any number of them, each moving at unit cost. Real machines do not extend that courtesy. A pointer that stays hot lives in registers and low cache levels; keep many of them hot and they compete for the same fast silicon. This exposition adds that competition to D as a **priced, structural** resource — no cache simulator, no eviction model: one inequality and one surcharge.

1 The objects

Definition (touch sequence). A **touch sequence** over k heads is a finite word $t = t_1 \dots t_n$ with $t_j \in \{1, \dots, k\}$: the j -th operation touches head t_j . Write c_i for the number of touches of head i , so $\sum_i c_i = n$.

Definition (distance assignment). A **distance assignment** is a function $d : \{1, \dots, k\} \rightarrow \mathbb{N}$; d_i is the depth at which head i is mounted in an implicit binary **fast-state tree**. Its **Kraft mass** is $\sum_i 2^{-d_i}$, and d is **admissible** when the mass is at most 1.

Definition (mounting). A **mounting** realizing d assigns heads injectively to binary paths, head i to a path of length exactly d_i , any two paths prefix-incomparable. Prefix-incomparability is dyadic disjointness: the subtrees below two mount slots never overlap, so shares cannot alias. No mounted tree is ever walked — the distance is the address; the tree exists so that “only so much fits nearby” is a theorem rather than a metaphor.

Definition (touch cost). Under an assignment d , a touch sequence costs

$$\text{touchCost}(d, t) = \sum_{j=1}^n (1 + d_{t_j}) = n + \sum_i c_i d_i :$$

every touch pays its unit plus the walk to its head’s slot.

Definition (empirical distances and entropy). For a sequence with every $c_i > 0$, the **empirical (inverse-frequency) distances** are $d_i = \lceil \log_2(n/c_i) \rceil$. The **empirical entropy**, in count form, is $\sum_i c_i \log_2(n/c_i)$ — that is $n \cdot H(\hat{p})$ for the empirical frequencies $\hat{p}_i = c_i/n$.

None of these carries a receipt: definitions claim nothing, they set the stage (AIP-4). Every **claim** about them below carries its badge.

2 The model

Two rules give the objects their teeth (cache-v0, after convo [55]–[59]):

1. **Budget.** Only admissible assignments are allowed. Only so much fits nearby: head i occupies the fraction 2^{-d_i} of fast state, and the shares are disjoint by prefix-incomparability — overlap is what makes sharing models treacherous.
2. **Surcharge.** Every paid operation of head i costs $1 + d_i$; up remains free because it acquires no address bit. A hot head is near and therefore cheaper to touch.

The plain machine is the fiber $d \equiv 0$:

LEAN $\vdash$ Adic.Dyadic.distCost_zero

Distance is a refinement

At distance zero everywhere, cost is exactly the old cost — every Expo 1 theorem survives as the $d \equiv 0$ fiber, and cost remains a homomorphism on words.

```
theorem distCost_zero {k : Nat} (word : ActionWord k) :
  distCost (fun x => 0) word = actionCost word
```

axioms: propext hash: 86a107f6f54c

The Kraft condition is not an analogy but the region algebra itself: admissible distances are exactly antichains of dyadic mount slots.

LEAN $\vdash$ Adic.Dyadic.kraft_iff_mounting

Kraft = an antichain of mount slots

A distance assignment satisfies Kraft **iff** the heads can be mounted injectively on pairwise prefix-incomparable paths with $|\text{path}_i| = d_i$ — prefix-free codes, dyadic disjointness, and fast-state budgeting are one condition. Both directions, with a constructive canonical mounting.

```
theorem kraft_iff_mounting {k : Nat} (dists : Dists k) :
  KraftOk dists ↔
    ∃ paths,
      (∀ (head : Fin k), (paths head).length = dists head) ∧
      Function.Injective paths ∧
      ∀ {left right : Fin k},
        left ≠ right → PrefixIncomparable (paths left) (paths right)
```

axioms: Classical.choice, Quot.sound, propext hash: 9f2039740bce

3 Streaming survives distance

LEAN $\vdash$ Adic.Dyadic.distZipCost_closed

Zip at a distance, exact

The zip schedule of Expo 1 under fixed distances has an exact closed-form cost, still linear in 2^n — the paid leaf and node coefficients expose the surcharge exactly, while free ascents contribute nothing. The convo’s own worked example ([59]) — three heads at uniform distance 2, Kraft mass $\frac{3}{4}$ — is instantiated and checked:

```
theorem distZipCost_closed (dists : Dists 3) (n : Nat) :
  distZipCost dists n =
    (6 + dists inputAHead + dists inputBHead + 4 * dists outputHead) * 2 ^ n +
    (6 + 2 * (dists inputAHead + dists inputBHead + dists outputHead)) *
    (2 ^ n - 1)
```

axioms: Quot.sound, propext hash: aa03139efba5

LEAN $\vdash$ Adic.Dyadic.uniformThreeDists_kraft

...and [59]’s example is admissible

$3 \cdot 2^{-2} \leq 1$, in the all-natural formulation, no real numbers anywhere.

```
theorem uniformThreeDists_kraft : KraftOk uniformThreeDists
```

axioms: Quot.sound, propext hash: 8e9b9e91f167

4 The entropy bound

Which distances should a program choose? Count the touches: head i touched c_i times out of n . Choosing distances against inverse empirical frequency is admissible and essentially optimal — the fast-state allocation problem is source coding.

LEAN $\vdash$ Adic.Dyadic.empiricalDists_kraft

Inverse-frequency distances are admissible

The inverse-frequency choice $d_i = \lceil \log_2(n/c_i) \rceil$ satisfies Kraft — the classical prefix-code argument, in $\mathbb{N}$.

```
theorem empiricalDists_kraft {k : Nat} (touches : List (Fin k))
  (hnonempty : 0 < touches.length)
  (hpositive : ∀ (head : Fin k), 0 < touchCount touches head) :
  KraftOk (empiricalDists touches)
```

axioms: Quot.sound, propext hash: 29c28abfbdf7

LEAN $\vdash$ Adic.Dyadic.touchCost_empirical_entropy_bound

Achievability: cost $\leq$ entropy + slack

Under those distances the total touch cost is bounded by the empirical entropy of the touch sequence (count form $\sum_i c_i \log(n/c_i)$) plus explicit linear slack — the $H(p) + O(1)$ per touch of convo [57], with every ceiling accounted.

```
theorem touchCost_empirical_entropy_bound {k : Nat} (touches : List (Fin k))
  (hnonempty : 0 < touches.length)
  (hpositive : ∀ (head : Fin k), 0 < touchCount touches head) :
  touchCost (empiricalDists touches) touches ≤
    empiricalFloorLogTotal touches + 2 * touches.length
```

axioms: Quot.sound, propext hash: e49667460df8

LEAN $\vdash$ Adic.Dyadic.touchCost_entropy_lower

Optimality: no distances beat entropy

Conversely, **every** Kraft-admissible assignment pays at least the empirical entropy (floor form, explicit slack): Shannon’s lower bound, carried by a discrete Gibbs/log-sum inequality proved by the counting route —

```
theorem touchCost_entropy_lower {k : Nat} (dists : Dists k)
  (touches : List (Fin k)) (hKraft : KraftOk dists) :
  touches.length + empiricalFloorLogTotal touches ≤
    touchCost dists touches + touches.length
```

axioms: Quot.sound, propext hash: 3e2e47c38273

LEAN $\vdash$ Adic.Dyadic.discrete_gibbs_log_sum

The discrete log-sum inequality

$\sum_i c_i \lceil \log_2(n/c_i) \rceil \leq \sum_i c_i d_i + n$ for any Kraft-admissible d — all-natural, zero-count heads included.

```
theorem discrete_gibbs_log_sum {k : Nat} (total : Nat)
  (counts : Fin k → Nat) (dists : Dists k) (htotal : finSum counts = total)
  (hKraft : KraftOk dists) :
  floorLogTotal total counts ≤ distCountTotal counts dists + total
```

axioms: Quot.sound, propext hash: 21aec34cbd41

Together: fast state allocated like a prefix code, priced like a code length, optimal like a Shannon code. Biased search trees (Bent–Sleator–Tarjan) are the data-structure ancestor; adaptive distance assignment via dynamic Huffman/FGK reweighting is a known road, deferred.

5 What this model refuses

DESK-PROVED — not mechanized

Kraft pricing is not cliff pricing

k -way merge under Kraft distances costs $n \log n$ for **every** k — the multi-way tradeoff cancels exactly ([57]’s honest computation). The external-memory win (log base $\frac{M}{B}$) comes from the capacity **cliff**, which this model deliberately lacks: it never under-charges, and pessimism is the right failure mode for a verifier. Mechanizing this cancellation is phase 3; until then it is desk-level.

UNFORMALIZED

The write-head distance ([72])

Should zip’s write head, moving twice the data, get a larger share? Touch-frequency pricing says no (all three heads touch equally often); data-rate pricing says yes. The tension is exactly touch-frequency distances vs size-aware distances, and its resolution belongs to the size-aware distance refinement (v1), where a head’s state size enters the budget.